Пројектовање синтаксног анализатора 3

- Ручно: анализатор са рекурзивним спуштањем
- Уз помоћ алата: сличност са интерпретером

Синтаксни анализатор са рекурзивним спуштањем (енгл. recursive-descent)

- Има по једну (међурекурзивну) функцију за сваки нетерминал (синтаксну класу)
  - Унутар сваке функције switch по врсти улазног симбола
    - са по једном case клаузулом за сваку продукцију
- Синоним: анализа са предвиђањем (predictive)
- Ограничење:
  - Ради добро само ако први терминални симбол у сваком подизразу једнозначно одређује продукцију
    - Ако то није случај, постоји конфликт

Пример 1 (1/3)
Спецификација граматике

- Направити синтаксни анализатор са рекурзивним спуштањем за задату граматику
  - Нетерминални симболи: S, E, L
    - Дефинишу дозвољене исказе и изразе у језику
  - Терминали: let, in, begin, print, end, “;”, “=”, num
    - Кључне речи и оператори у језику

S → let E in S
S → begin S L
S → print E

L → end
L → ; S L
E → num = num

Пример 1 (2/3)

Програм

$S \rightarrow$ let E in S
$S \rightarrow$ begin S L
$S \rightarrow$ print E

$L \rightarrow$ end
$L \rightarrow$; S L
$E \rightarrow$ num = num

```
enum token {LET, IN, BEGIN, END, PRINT, SEMI, NUM, EQ};
enum token getToken();

enum token tok;
void advance() { tok = getToken(); }
void eat(enum token t) {
  if (tok == t) advance();
  else error();
}

void S() {
  switch (tok) {
    case LET:   eat(LET); E(); eat(IN); S(); break;
    case BEGIN: eat(BEGIN); S(); L(); break;
    case PRINT: eat(PRINT); E(); break;
    default: error();
  }
}
```

Пример 1 (3/3)

Програм

$S \rightarrow$ let E in S
$S \rightarrow$ begin S L
$S \rightarrow$ print E

$L \rightarrow$ end
$L \rightarrow$; S L
$E \rightarrow$ num = num

```
void L() {
  switch (tok) {
    case END:  eat(END); break;
    case SEMI: eat(SEMI); S(); L(); break;
    default: error();
  }
}

void E() {
  eat(NUM);
  eat(EQ);
  eat(NUM);
}

void main() {
  advance();
  S();
}
```

Примери са конфликтом

E → a b T
E → a c G

- Први терминални симбол не одређује продукцију.
  - Али одређују прва два
  - За неке граматике треба више од једног терминалног симбола за предвиђање продукције
  - Није суштински проблем, парсер са рекурзивним спуштањем се оваквом случају релативно лако прилагођава

Примери са конфликтом

$E \rightarrow a\ b\ T$
$E \rightarrow a\ c\ G$

- Први терминални симбол не одређује продукцију.

```
void E() {
  switch (tok) {
    case A_TOK:
      eat(A_TOK);
      switch (tok) {
        case B_TOK: eat(B_TOK); T(); break;
        case C_TOK: eat(C_TOK); G(); break;
      }
    break;
  }
}
```

Примери са конфликтом

E → a T x
E → a T z

- Први терминални симбол не одређује продукцију.
  - И не можемо знати коју продукцију да изаберемо пре нетерминала T
  - Формалније речено: FIRST(a T x) се преклапа са FIRST(a T y)
  - За низ терминала и нетерминала γ (реченична форма), скуп FIRST чине сви терминали којима започињу реченице развијене из γ
    - Нпр. за γ = T + F у некој граматици, FIRST(γ) = {id, num, (}

Примери са конфликтом

E → T x
E → K z

- Први терминални симбол не одређује продукцију.
  - Сличан проблем...
  - ...али само ако се FIRST(T x) преклапа са FIRST(K y)

Примери са конфликтом

E → a T x
E → a T z

E → a T E′
E′ → x
E′ → z

Решење конфликта

- Први терминални симбол не одређује продукцију.
  - И не можемо знати коју продукцију да изаберемо пре нетерминала T
  - Формалније речено: FIRST(a T x) се преклапа са FIRST(a T y)
  - За низ терминала и нетерминала γ (реченична форма), скуп FIRST чине сви терминали којима започињу реченице развијене из γ
    - Нпр. за γ = T + F у некој граматици, FIRST(γ) = {id, num, (}

Примери са конфликтом

E → a T x
E → a T

E → a T E′
E′ → x
E′ → ε

Решење конфликта

- Слично као мало пре
  - Симбол E′ је поништив (nullable) ако се из њега може развити празан низ
  - Сада нам треба и FOLLOW(T) скуп, да бисмо знали да ли да применимо редукцију која поништава неки нетерминални симбол
  - Скуп FOLLOW(X) је скуп терминала који у реченицама језика могу пратити нетерминал X

Примери са конфликтом

E → a T b c
T → b
T → ε

- Не можемо разликовати прву и другу продукцију T
  - FIRST(T) и FOLLOW(T) скуп се преклапају

Примери са конфликтом

E → a T b c
T → b
T → ε

E → a b b c
E → a b c

Решење конфликта

- Не можемо разликовати прву и другу продукцију T
  - FIRST(T) и FOLLOW(T) скуп се преклапају
  - Решење конфликта може увести нови конфликт (али другачије врсте)

Алгоритам за одређивање

FIRST, FOLLOW и NULLABLE

Initialize FIRST and FOLLOW to empty sets, and nullable to all false.
for each terminal symbol Z
FIRST[Z] = {Z}
repeat
for each production $X \rightarrow Y_1\ Y_2 \ldots Y_k$
for each i from 1 to k, each j from i+1 to k
if all the Yi are nullable
then nullable[X] ← true
if $Y_1 \ldots Y_{i-1}$ are all nullable
then FIRST[X] ← FIRST[X] U FIRST[Y_i]
if $Y_{i+1} \ldots Y_k$ are all nullable
then FOLLOW[Y_i] ← FOLLOW[Y_i] U FOLLOW[X]
if $Y_{i+1} \ldots Y_{j-1}$ are all nullable
then FOLLOW[Y_i] ← FOLLOW[Y_i] U FOLLOW[Y_j]
until FIRST, FOLLOW, and nullable did not change in this iteration.

Пример са конфликтима

S → E $

E → E + T	**T → T * F**	**F → id**
E → E - T	**T → T / F**	**F → num**
E → T	**T → F**	**F → (E)**

- Лева рекурзија
  - Специјалан случај преклапања FIRST скупова две продукције

Уклањање леве рекурзије

- Опште правило за преписивање израза са левом рекурзијом у изразе са десном рекурзијом

Početni skup produkcija	Rezultat transformacije
$X \rightarrow X\,\gamma_1$	$X \rightarrow \alpha_1\,X'$
$X \rightarrow X\,\gamma_2$	$X \rightarrow \alpha_2\,X'$
$X \rightarrow \alpha_1$	$X' \rightarrow \gamma_1\,X'$
$X \rightarrow \alpha_2$	$X' \rightarrow \gamma_2\,X'$
	$X' \rightarrow$

α - не садржи X

γ - не почиње са X

Језик након трансформације:

$S \rightarrow E\ \$$

$E \rightarrow T\ E'$

$E' \rightarrow + T\ E'$

$E' \rightarrow - T\ E'$

$E' \rightarrow$

$T \rightarrow F\ T'$

$T' \rightarrow * F\ T'$

$T' \rightarrow / F\ T'$

$T' \rightarrow$

$F \rightarrow id$

$F \rightarrow num$

$F \rightarrow (\ E\ )$

Садржаји скупова

- Поред трансформације језика, потребно је одредити садржаје скупова
  - За сваки нетерминал: nullable, FIRST и FOLLOW

Симбол	nullable	FIRST	FOLLOW
S	false	(id num	
E	false	(id num	) $
E'	true	+ -	) $
T	false	(id num	) + - $
T'	true	* /	) + - $
F	false	(id num	) * / + - $

Интерпретер са рекурзивним спуштањем (1/2)

$F \rightarrow id$
$F \rightarrow num$
$F \rightarrow (E)$

```
enum token {EOF, ID, NUM, PLUS, MINUS, ...};
union tokenval {string id; int num; ...};
enum token tok;
union tokenval tokval;
int lookup(string id) {...}

int F() {
  switch (tok) {
    case ID: {int i = lookup(tokval.id); advance(); return i;}
    case NUM: {int i = tokval.num; advance(); return i;}
    case LPAREN: { eat(LPAREN);
                   int i = E();
                   eat(RPAREN);
                   return i; }
    case EOF:
    default:  printf("Expected ID, NUM, or (");

              return 0;
  }
}
```

Интерпретер са рекурзивним спуштањем (1/2)

$F \rightarrow id$
$F \rightarrow num$
$F \rightarrow (E)$

```
enum token {EOF, ID, NUM, PLUS, MINUS, ...};
union tokenval {string id; int num; ...};
enum token tok;
union tokenval tokval;
int lookup(string id) {...}
int F_follow[] = {RPAREN, TIMES, DIVIDE, PLUS, MINUS, EOF, -1};
int F() {
  switch (tok) {
    case ID: {int i = lookup(tokval.id); advance(); return i;}
    case NUM: {int i = tokval.num; advance(); return i;}
    case LPAREN: { eat(LPAREN);
                   int i = E();
                   eatOrSkipTo(RPAREN, F_follow);
                   return i; }
    case EOF:
    default:  printf("Expected ID, NUM, or (");
              skipto(F_follow);
              return 0;
  }
}
```

Интерпретер са рекурзивним спуштањем (2/2)

$T \rightarrow F\ T'$
$T' \rightarrow *\ F\ T'$
$T' \rightarrow /\ F\ T'$
$T' \rightarrow$

```
int T() {
  return Tprim(F());
}
```

```
int Tprim(int a) {
  switch (tok) {
    case TIMES: eat(TIMES); return Tprim(a * F());
    case DIVIDE: eat(DIVIDE); return Tprim(a / F());
    case RPAREN: case PLUS: case MINUS: case EOF: return a;
    default: ...
  }
}
void eatOrSkipTo(int expected, int* stop) {
  if (tok == expected) eat(expected);
  else {printf(...); skipto(stop);}
}
```

FOLLOW(T')

Originalne produkcije:
$T \rightarrow T * F$
$T \rightarrow T / F$
$T \rightarrow F$
T* se prenosi kroz *a

Интерпретер са рекурзивним спуштањем (2/2)

$T \rightarrow F\ T'$
$T' \rightarrow *\ F\ T'$
$T' \rightarrow /\ F\ T'$
$T' \rightarrow$

```
int T_follow[] = {RPAREN, PLUS, MINUS, EOF, -1};
int T() {
  switch (tok) {
    case ID: case NUM: case LPAREN: return Tprim(F());
    default: printf("Expected ID, NUM, or (");
             skipto(T_follow);  return 0;
  }
}
int Tprim(int a) {
  switch (tok) {
    case TIMES: eat(TIMES); return Tprim(a * F());
    case DIVIDE: eat(DIVIDE); return Tprim(a / F());
    case RPAREN: case PLUS: case MINUS: case EOF: return a;
    default: ...
  }
}
void eatOrSkipTo(int expected, int* stop) {
  if (tok == expected) eat(expected);
  else {printf(...); skipto(stop);}
}
```

FIRST(F)

FOLLOW(T')

Originalne produkcije:

$T \rightarrow T * F$
$T \rightarrow T / F$
$T \rightarrow F$

T* se prenosi kroz *a

Напомене уз претходни Це код

- Семантика
  - Симболи ID и NUM носе вредности типа sting и int
  - Подразумева се постојање табеле
    - Која пресликава ID-ове на NUM-ове (целе бројеве)
  - Тип придружен нетерминалима E, T, F, итд. је int
    - Семантичке акције се лако реализују
- Проблем: Семантичка акција за вештачки симбол T’
  - Нпр. у продукцији T’→*FT’ недостаје леви операнд за *
  - Један начин је да се леви операнд проследи као аргумент
    - Функција T га узима са F() и прослеђује функцији Tprime
- Развој симбола у празан стринг се препознаје појавом неког од симбола из скупа FOLLOW

Однос интерпретера и синтаксног анализатора

- Постоји велика сличност између њих
- Разликују се само акције
  - Интерпретер: акције доводе до директног извршења
    - Нпр. исказ доделе вредности заиста уписује вредност у променљиву
    - Исказ штампања (print) садржаја променљиве га заиста приказује на монитору, итд.
  - Синтаксни анализатор: акције изграђују стабло апстрактне синтаксе додавањем нових чворова
    - То стабло ће касније у низу трансформација бити преведено у машински код
    - Тек при његовом извршење добијају се резултати

Граматика праволинијских програма

- SLP интерпретер и синтаксни анализатор за SLP
  - Граматика SLP задата доњим низом продукција
- Преузимањем структура из ручно писаног SLP интерпретера, долази се до Bison(Yacc) спецификација
  - У случају синтаксног анализатора те структуре података представљају апстрактну синтаксу

S → S; S
S → id := E
S → print (L)

E → id
E → num
E → E B E
E → S, E
E → (E)

L → E
L → L, E

B → + | - | * | /

Спецификација SLP интерпретера (1/2)

```
%{
#include "table.h"
extern table tab;
%}

%union {int ival; string sval;}
%token <sval> ID
%token <ival> INT
%token ASSIGN PRINT LPAREN RPAREN
%type <ival> exp

%right SEMICOLON
%left MINUS PLUS
%left TIMES DIVIDE
%start prog

%%
prog:  stm
```

Спецификација SLP интерпретера (2/2)

```
stm:  stm SEMICOLON stm
stm:  ID ASSIGN exp                          {update(&tab, $1, $3);}
stm:  PRINT LPAREN exps RPAREN  {printf("\n");}

exps: exp                                    {printf("%d ", $1);}
exps: exps COMMA exp                 {printf("%d ", $3);}

exp:  INT                                    {$$ = $1;}
exp:  ID                                     {$$ = lookup(tab, $1);}
exp:  exp PLUS exp                   {$$ = $1 + $3;}
exp:  exp MINUS exp                  {$$ = $1 - $3;}
exp:  exp TIMES exp                  {$$ = $1 * $3;}
exp:  exp DIVIDE exp                 {$$ = $1 / $3;}
exp:  stm COMMA exp                  {$$ = $3;}
exp:  LPAREN exp RPAREN  {$$ = $2;}

// $1, $2, itd. se odnose na definicione članove produkcije
```

Спецификација SLP синтаксног анализатора (1/2)

```
%{
extern A_stm rootprg;
%}

%union {int num; string id; IrStm* stm; IrExp* exp; IrExpList* expList;}
%token <num> INT
%token <id> ID
%token ASSIGN PRINT LPAREN RPAREN
%type <stm> stm prog
%type <exp> exp
%type <expList> exps
%left SEMICOLON
%left MINUS PLUS
%left TIMES DIVIDE
%start prog

%%
prog:  stm                {$$=$1; rootprg=$1;}
```

Спецификација SLP синтаксног анализатора (2/2)

```
stm:  stm SEMICOLON stm      {$$=makeCompoundStm($1, $3);}
stm:  ID ASSIGN exp          {$$=makeAssignStm ($1, $3);}
stm:  PRINT LPAREN exps RPAREN  {$$=makePrintStm ($3);}

exps: exp                    {$$=makeLastExpList($1);}
exps: exp COMMA exps         {$$=makePairExpList($1, $3);}

exp:  INT                    {$$=makeNumExp($1);}
exp:  ID                     {$$=makeIdExp($1);
exp:  exp PLUS exp           {$$=makeOpExp($1, OK_PLUS, $3);}
exp:  exp MINUS exp          {$$=makeOpExp($1, OK_MINUS, $3);}
exp:  exp TIMES exp          {$$=makeOpExp($1, OK_MUL, $3);}
exp:  exp DIVIDE exp         {$$=makeOpExp($1, OK_DIV, $3);}
exp:  stm COMMA exp          {$$=makeEseqExp($1, $3);}
exp:  LPAREN exp RPAREN      {$$=$2;}
```

Мала илустрација још једног алгоритма за синтаксну анализу

- Помери и редукуј (енгл. shift-reduce) парсер (LR).
- Такви парсери се зову још и „Од доле ка горе“ (енгл. bottom-up) због начина на који се гради стабло синтаксне анализе.

1. S → E $ **2. E → T** **4. T → id**
3. E → E + T **5. T → (E)**

Пример рада LL парсера

(id + id)$	
	Правило
S	S -> E$
E$	E -> T
T$	T -> (E)
(E)$	E -> E + T
(E + T)$	E -> T
(T + T)$	T -> id
(id + T)$	T -> id
(id + id)$	

1. S → E $ **2. E → T** **4. T → id**
3. E → E + T **5. T → (E)**

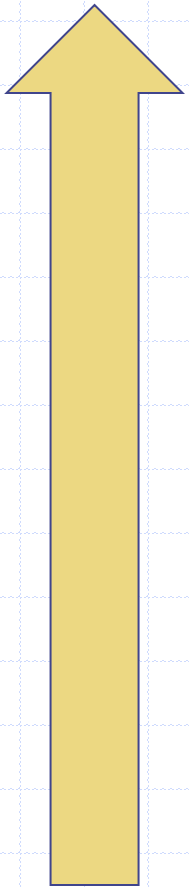
Пример рада LR парсера

Стек	Улаз	
	(id + id)$	Помери
(	id + id)$	Помери
(id	+ id)$	Редукуј: T -> id
(T	+ id)$	Редукуј: E -> T
(E	+ id)$	Помери
(E +	id)$	Помери
(E + id	)$	Редукуј: T -> id
(E + T	)$	Редукуј: E -> E + T
(E	)$	Помери
(E)	$	Редукуј: T -> (E)
T	$	Редукуј: E -> T
E	$	Помери
E$		Редукуј: S -> E$
S		

1. S → E $ **2. E → T** **4. T → id**
3. E → E + T **5. T → (E)**

Пример рада LR парсера

Стек	Улаз	
	(id + id)$	Помери
(	id + id)$	Помери
(id	+ id)$	Редукуј: T -> id
(T	+ id)$	Редукуј: E -> T
(E	+ id)$	Помери
(E +	id)$	Помери
(E + id	)$	Редукуј: T -> id
(E + T	)$	Редукуј: E -> E + T
(E	)$	Помери
(E)	$	Редукуј: T -> (E)
T	$	Редукуј: E -> T
E	$	Помери
E$		Редукуј: S -> E$
S		



Крајњи десни извод

Општи облик редукција (1/2)

LABELA:	PRETHODNI VRH STEKA	RUTINA AKCIJA	NOVI VRH STEKA	SLEDEĆA REDUKCIJA

- Лабела је опциона
- Претходни врх стека
  - Изостављен – увек се поклапа
  - Симбол
    - СИНТАКСНИ као што је идентификатор или литерал - поклапа се са неким симболом овог типа
    - било који (*) - поклапа се са симболом било ког типа
    - симболички - представљена кључна речи као “WHILE" или "IF"

Општи облик редукција (2/2)

LABELA:	PRETHODNI VRH STEKA	RUTINA AKCIJA	NOVI VRH STEKA	SLEDEĆA REDUKCIJA

- Рутина акција – назив рутине акције, тј. синтаксне акције (опционо)
- Нови врх стека:
  - знак празнине или нула - нема промене
  - одстранити врх стека (поворку која се поклапа)
  - синтаксни тип, кључна реч или елеменат стека - заменити претходни врх стека са овим елементом
  - узети следећи униформни симбол из табеле униформних симбола и ставити га на врх стека